

Enhancing repository-level software repair via repository-aware knowledge graphs

BOYANG YANG, JIADONG REN, and SHUNFU JIN, Yanshan University, China
 YANG LIU, Nanyang Technological University, Singapore
 FENG LIU and BACH LE, University of Melbourne, Australia
 HAOYE TIAN*, Aalto University, Finland

Repository-level software repair faces challenges in bridging semantic gaps between issue descriptions and code patches. Existing approaches, which primarily rely on large language models (LLMs), are hindered by semantic ambiguities, limited understanding of structural context, and insufficient reasoning capabilities. To address these limitations, we propose KGCOMPASS with two innovations: (1) a novel repository-aware knowledge graph (KG) that accurately links repository artifacts (issues and pull requests) and codebase entities (files, classes, and functions), allowing us to effectively narrow down the vast search space to only 20 most relevant functions with accurate candidate fault locations and contextual information, and (2) a path-guided repair mechanism that leverages KG-mined entity paths, tracing through which allows us to augment LLMs with relevant contextual information to generate precise patches along with their explanations. Experimental results in the SWE-bench Lite demonstrate that KGCOMPASS achieves state-of-the-art single-LLM repair performance (58.3%) and function-level fault location accuracy (56.0%) across open-source approaches with a single repair model, costing only \$0.2 per repair. Among the bugs that KGCOMPASS successfully localizes, 89.7% lack explicit location hints in the issue and are found only through multi-hop graph traversal, where pure LLMs struggle to locate bugs accurately. Relative to pure-LLM baselines, KGCOMPASS lifts the resolved rate by 50.8% on Claude-4 Sonnet, 30.2% on Claude-3.5 Sonnet, 115.7% on DeepSeek-V3, and 156.4% on Qwen2.5 Max. These consistent improvements demonstrate that this graph-guided repair framework delivers model-agnostic, cost-efficient repair and sets a strong new baseline for repository-level repair.

ACM Reference Format:

Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing repository-level software repair via repository-aware knowledge graphs. 1, 1 (October 2025), 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable coding capabilities in code generation and repair tasks [21, 25, 26, 34, 42, 51]. To evaluate capabilities at a realistic scale, SWE-bench benchmark compiles 2,294 issues with corresponding codebases and test suites from 12 Python repositories [16]. Because running the full benchmark is resource-intensive, the authors also released SWE-bench Lite, a curated subset of 300 tasks that maintains the original difficulty while reducing compute costs. As a result, SWE-bench Lite has become the most widely used benchmark

*Corresponding author.

Authors' Contact Information: Boyang Yang; Jiadong Ren; Shunfu Jin, Yanshan University, China; Yang Liu, Nanyang Technological University, Singapore; Feng Liu; Bach Le, University of Melbourne, Australia; Haoye Tian, Aalto University, Finland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2025/10-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

for repository-level repair tasks [32, 35, 39, 41]. Recent research on these benchmarks shows that the biggest challenge for LLM-based repair is to precisely pinpoint one or a few buggy functions hidden among a repository including thousands of files and more functions [16, 46]. Imprecise fault location propagates to patch generation and depresses end-to-end repair accuracy.

To address these repository-level repair challenges, researchers have explored two main categories of approaches: agentic [1, 12, 18, 39, 45] and procedural [20, 32, 41]. Agentic approaches equip LLMs with various tools for autonomous planning and execution through multiple specialized agents, but lack interpretable reasoning and controlled decision planning. Procedural approaches mitigate these issues through expert-designed workflows, offering more precise and interpretable processes [32, 41]. A central bottleneck of procedural approaches is the selection of candidate locations provided to the patch generation. Systems such as Agentless [41] pass only one file location per repair; if that prediction is wrong, the repair often fails. Supplying more candidates improves recall but inflates context, increases cost, and raises hallucination risk for long inputs, which degrades patch accuracy [48]. Consequently, the existing approaches still face two major limitations:

- ① **Imprecise Fault Location:** Precise fault location at repository scale requires cross-modal grounding that aligns natural-language bug reports to codebase entities and links related issues and pull requests to those entities. In SWE-bench Lite, only about 32.7% of issue descriptions of instances explicitly include the name of the file or the function of the ground truth location. Yet, most issues mention at least one existing repository identifier, such as an issue or PR number or a file path, which can serve as an anchor. Existing KG-based repair systems emphasize text analysis or intra-code structure and leave this alignment underspecified, such as RepoGraph [33]. Procedural repair systems like Agentless [41] also commit to a single predicted location rather than fusing corroborated locations across multiple files that supply repair context. This missing grounding makes precise pinpointing difficult and lowers repair accuracy on large codebases [16].
- ② **Lack of Decision Interpretability:** Repository-level software repair requires traceable reasoning chains to validate the repair decisions [6]. While existing state-of-the-art approaches have achieved promising results, they lack interpretable decision processes in fault location and patch generation [37]. Current agentic approaches [1, 39, 45] often make decisions through complex black-box interactions without clear reasoning paths, while procedural approaches [32, 41] rank multiple candidate patches but provide limited insight into their final choices for users. On SWE-bench Lite, while state-of-the-art repair systems achieve more than 60.0% repair accuracy, they fail to provide transparent reasoning information, especially for fault location, limiting both trustworthiness and practical application.

This paper. We present KGCOMPASS, a novel approach that accurately links code structure with repository metadata into a unified database, namely a knowledge graph, to achieve more accurate and interpretable software repair. Given a natural language bug report and a repository codebase, our approach constructs a knowledge graph that captures relationships between repository artifacts (issues and pull requests) and codebase entities (files, functions, classes). This unified representation allows us to effectively trace the semantic connection between bug descriptions and potential fault locations, reducing the search space from thousands to 20 most relevant candidates that accurately provide both fault locations and contextual information. The useful contextual information provided by the knowledge graph is then used to help LLMs generate more accurate patches.

Experimental results show that KGCOMPASS demonstrates state-of-the-art single-LLM repair performance on SWE-bench Lite at a cost of only \$0.2 per repair. With Claude-4 Sonnet, KGCOMPASS attains 58.3% repair accuracy, a 50.8% relative gain over the pure-LLM baseline. KGCOMPASS also lifts DeepSeek-V3 by 115.7% and Qwen2.5 Max by 156.4% relative to their respective pure-LLM baselines,

demonstrating the generalizability of KGCOMPASS across diverse backbone LLMs. KGCOMPASS further achieves superior 83.6% file-level and 56.0% function-level fault location accuracy and provides clear reasoning chains that users can inspect. Analysis shows that 89.7% of ground-truth patches require multi-hop connections, underscoring the value of exploiting indirect relationships by KG. Ablation studies confirm the individual contribution of each component.

Our main contributions are:

- **Repository-Aware Knowledge Graph:** We present a technique for constructing a repository-aware knowledge graph that merges code entities (files, classes, and functions) with issue and pull-request artifacts, thus capturing both structural dependencies and semantic relations across the entire repository.
- **KG-based Repair Approach:** We introduce KGCOMPASS, a KG-guided repair framework that supplies an LLM with rich structural and semantic context from KG, enabling it to locate bugs precisely and synthesize accurate patches at repository scale.
- **Efficient Search Space Reduction:** KGCOMPASS lowers the cost to only \$0.2 per repair by using KG-based candidate selection to shrink the function search space from thousands to just 20. Top 20 candidates cover 91.0% of file-level and 66.7% of function-level ground truth fault locations.
- **Comprehensive Experimental Evaluation:** Results on SWE-bench Lite demonstrate KGCOMPASS’s highest repair accuracy (58.3%) and function-level fault location accuracy (56.0%) among all open-source approaches based on Claude-4 Sonnet [32, 35, 39, 41]. Compared with pure-LLM baselines, KGCOMPASS yields improvements of 50.8% on Claude-4 Sonnet, 30.2% on Claude-3.5 Sonnet, 115.7% on DeepSeek-V3, and 156.4% on Qwen2.5 Max, confirming the generalization of KGCOMPASS across different backbone LLMs. Further ablation studies confirm the effectiveness of each component in our approach.

2 Motivating Example

We present two motivating examples from SWE-bench Lite [16], a benchmark for evaluating repository-level software repair. The first example illustrates KG-based fault location, and the second example illustrates KG-guided repair.

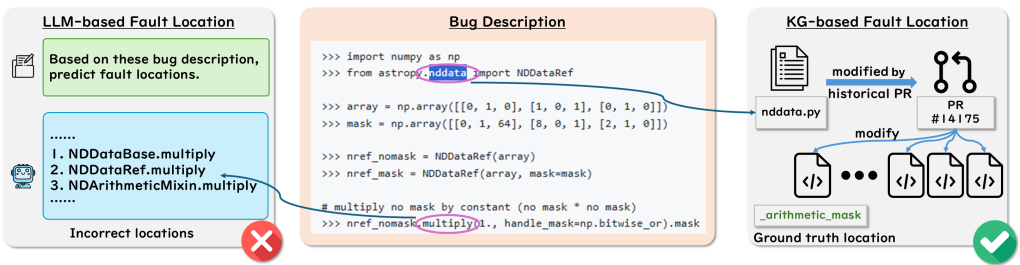


Fig. 1. Motivating Example of KG-based Fault Location

Knowledge Graph-guided Fault Location. As shown in Figure 1, instance astropy-14995 in SWE-bench Lite provides no explicit reference to the modified function within the ground truth patch, which only edits “NDArithmeticMixin._arithmetic_mask”. Conditioned only on the problem statement, LLM proposed three candidate locations that all revolve around a “multiply” routine because that token appears in the report, but none match the actual location.

Using the knowledge graph, we start from the issue text, detect the mention of “nddata”, resolve it to the repository file “nddata.py”, follow its link to pull request #14175, and arrive at “NDArithmeticMixin._arithmetic_mask”, which that PR modified. Our entity relevance scoring places this

function within the top five, which is sufficient for the downstream patch generation stage. This case shows how graph-derived multi-hop context counteracts lexical bias and recovers a correct location that a pure LLM misses, consistent with our KG mining and scoring strategy.

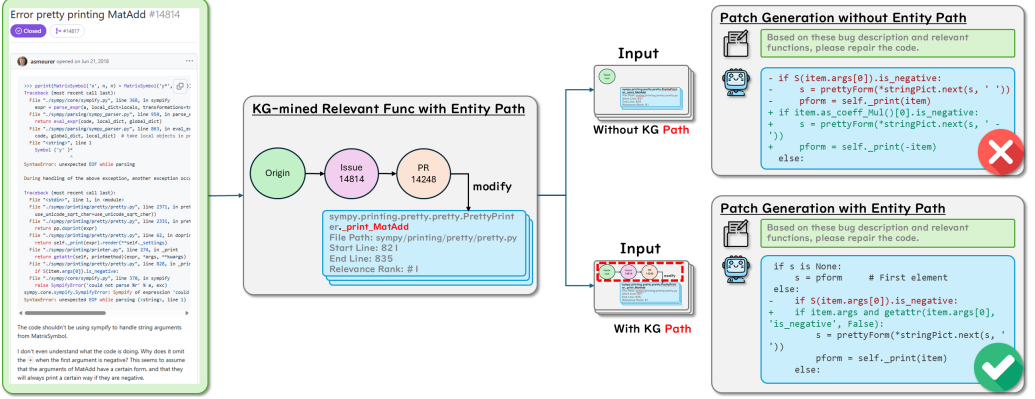


Fig. 2. Motivating Example of KG-guided Repair

Knowledge Graph-guided Repair. As shown in Figure 2, for the instance sympy-14814 of SWE-bench Lite, after the function “_print_MatAdd” is correctly identified, LLM produces a weak fix using “item.as_coeff_Mul()[0].is_negative” that fails when processing expressions with special characters like “y*”. However, when given the entity path (issue root → #14814 → #14248 → “_print_MatAdd”) from the knowledge graph, the LLM generates a robust solution using “getattr(item.args[0], 'is_negative', False)”. The contextual information associated with this path includes comments in issue #14814 referencing PR #14248, suggesting that “_print_MatAdd” should use the same functions as “_print_Add” for handling plus or minus signs. This contextual information guides the LLM to adopt a more defensive programming approach rather than directly accessing attributes that might not exist. The “getattr” solution safely handles cases where the “is_negative” attribute is unavailable, addressing the core weakness in the previous implementation. This case demonstrates how structural context from the knowledge graph helps LLMs identify relevant code patterns and relationships that would be missed when analyzing isolated code segments.

3 Approach

We present KGCOMPASS, a repository-level software repair framework that constructs a knowledge graph, enabling LLMs to locate accurate candidate fault locations and generate correct patches. Figure 3 illustrates our approach, which unfolds in three phases: knowledge graph mining, patch generation, and patch ranking.

In the **Knowledge Graph Mining phase**, (i) we construct a comprehensive knowledge graph by integrating multiple information sources from codebases and GitHub artifacts, including issues and pull requests. (ii) Then, we identify top candidate fault locations by extracting the 15 highest-scoring functions from the graph and (iii) augmenting them with up to 5 additional candidates suggested by the LLM. In the **Patch Generation phase**, (iv) we supply the LLM with these candidate fault locations, accompanied by corresponding source code and relevant entity paths within the graph structure. In the **Patch Ranking phase**, (v) we leverage an LLM to generate reproduction tests and (vi) utilize both LLM-generated reproduction tests and regression tests within codebases to evaluate and rank the generated candidate patches, then select the final patch that is most likely to resolve the target issue.

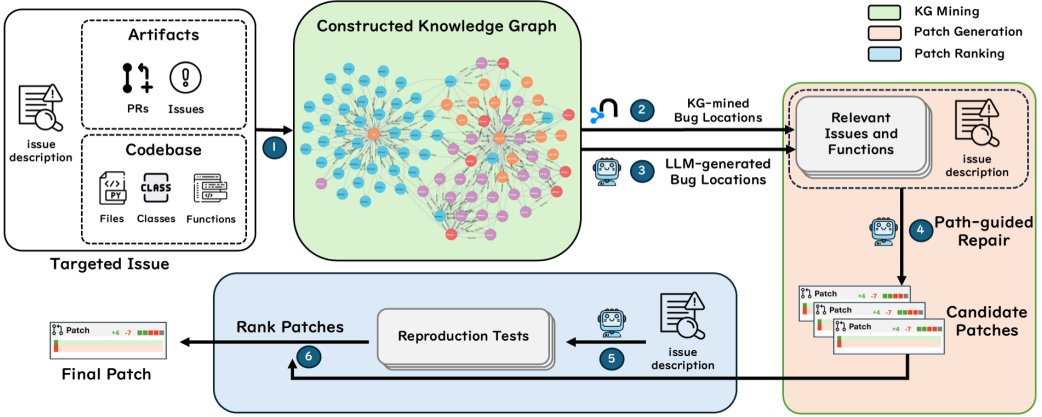


Fig. 3. Overview of KGCOMPASS

3.1 Knowledge Graph Mining

The Knowledge Graph mining phase constructs a comprehensive knowledge graph by analyzing problem statements to extract entities and their relationships, enabling the modeling of both structural and semantic connections that are critical for accurate fault location.

We organize the knowledge graph with two groups of nodes: *repository-level artifacts* (issues and pull requests) and *code-level entities* (files, classes, and functions). For code entities, we perform a static Abstract Syntax Tree (AST) analysis that (i) records hierarchical containment edges (file→class/function, class→function) and (ii) derives explicit reference edges, including *direct call relations* between functions, as well as import and attribute references. All such static dependencies are added to the knowledge graph as typed edges, providing the LLM with a rich structural subgraph that captures structural context such as calls, imports, attributes, and containment. For repository artifacts, we apply a template-based regular expression extractor to issue descriptions, comments, and pull request elements, identifying mentions of files, symbols, or line numbers and linking them back to the corresponding code nodes. To prevent data contamination from future artifacts, we include only artifacts whose timestamps precede the benchmark instance's `created_at` field, thereby mirroring the information realistically available to developers at report time. This filter rule removed 24.0% of the initially mined related artifacts. The built knowledge graph provides both fine-grained static program dependencies and cross-domain links between natural language descriptions and code, which later serve as informative paths for fault location and patch generation.

After constructing the knowledge graph, we compute each function entity's relevance score $S(f)$ through a novel formula that combines semantic similarity with structural proximity:

$$S(f) = \beta^{l(f)} \cdot (\alpha \cdot \cos_{\text{norm}}(e_i, e_f) + (1 - \alpha) \cdot \text{lev}(t_i, t_f)) \quad (1)$$

Where f represents a candidate function entity in the knowledge graph, e_i and e_f are embeddings of the problem description and function entity f . t_i and t_f are textual representations of the problem description and function entity f . $\text{lev}(\cdot, \cdot)$ is Levenshtein similarity normalized to $[0, 1]$, $\cos_{\text{norm}}(\cdot, \cdot)$ is cosine similarity normalized to $[0, 1]$, where $\cos_{\text{norm}} = \frac{\cos_{\text{raw}} + 1}{2}$. $l(f)$ is the weighted shortest path length from the issue node to function entity f using Dijkstra's algorithm [9], where a smaller path length indicates a closer connection. The hyper-parameter α controls the balance between semantic embedding similarity and textual similarity, following the finding of [7] that this combination effectively captures both contextual meaning and surface-level textual patterns. Lower

α values prioritize surface-level textual matches, which are crucial for identifying syntactically similar but semantically distinct code elements. The path length decay factor β determines how quickly relevance decreases with path distance through the exponential decay function $\beta^{l(f)}$. This function, adopted from prior work [50], effectively models relevance decay with graph distance while preserving meaningful multi-hop relationships.

Based on these relevance scores $S(f)$, we select the top 15 functions from the knowledge graph to provide candidate fault locations and context information. We then use an LLM to identify up to 5 potential fault locations from the problem statement, complementing candidate locations with the LLM's textual understanding. This hybrid configuration combines the strengths of knowledge graph-based structural analysis and LLM-based textual understanding, creating a comprehensive set of up to 20 candidate function-level fault locations for the following patch generation phase.

```
Based on the following bug description, predict potential relevant code locations:
Description:
{{problem_statement}}
Please provide a JSON array containing the predicted locations where the bug fix is needed.
Each location should include the full file path and the full function name.
The function field should be one of these formats:
- `package.module.Class.function_name` # For class's functions
- `package.module.function_name` # For standalone functions
- `package.module.Class.class_level_attribute` # For class-level attributes
- `package.module.MODULE_LEVEL_VALUE` # For module-level variables
Format:
[
  {
    "file_path": "package/submodule/file.py",
    "function": "package.module.Class.function_name"
  },
  {
    "file_path": "package/other/file.py",
    "function": "package.module.MODULE_LEVEL_VALUE"
  }
]
```

Fig. 4. LLM Prompt Template for Fault Location

3.2 Patch Generation

The patch generation phase leverages the identified candidates and graph context to generate patches. KGCOMPASS augments the LLM prompt with path information that illustrates structural connections (function calls, containment relationships, and references from repository artifacts). The prompt provided to the LLM consists of three key components: (1) the issue description, (2) KG-mined relevant functions with entity paths (formatted as Figure 5), and (3) a structured format specifying how to produce search/replace edits [13, 41]. The entity path explicitly traces connections from relevant functions to the targeted issue through intermediate entities like files, functions, issues, and pull requests. Incorporating KG-mined context enables KGCOMPASS to reason about the role and relevance of each relevant function.

To address LLMs' inherent context limitations and balance precision with diversity, KGCOMPASS employs a hybrid sampling strategy, combining deterministic (temperature = 0.0) and exploratory (temperature > 0.0) sampling during patch generation. Deterministic sampling ensures stability and consistency, while exploratory sampling introduces diversity to cover alternative viable repair scenarios.

Generated patches commonly suffer from syntactic errors due to incorrect indentation [24]. To address this, KGCOMPASS implements an adaptive indentation correction algorithm (Algorithm 1)


```

### [full file path]
- signature: [namespace].[class].[function name]([params])
- relationship_path: [function reference] --([relation type])--> [entity 1] --([relation type
])--> [entity 2] ... --> root
- start_line: [start]
- end_line: [end]
...
    [function code]
...

```

Fig. 5. KG-mined Relevant Function Format with Entity Path for Bug Repair

that systematically tests minor indentation adjustments (± 1 , ± 2 levels) and selects syntactically valid variants, recovering 2% of invalid patches. All syntactically valid patches are then passed to the subsequent patch ranking phase for evaluation through regression and reproduction tests.

Algorithm 1 Adaptive Indentation Correction Strategy

```

1: INPUT: Generated patch  $P$ , Original code  $S$ 
2: OUTPUT: Syntactically valid patch  $P'$  or null
3:  $P' \leftarrow \text{APPLYPATCH}(P, S)$ 
4: if  $\text{ISSYNTAXVALID}(P')$  and  $P' \neq S$  then
5:   return  $P'$ 
6: end if
7:  $\text{IndentLevels} \leftarrow \{-1, 1, -2, 2\}$ 
8: for  $i \in \text{IndentLevels}$  do
9:    $P_i \leftarrow \text{ADJUSTINDENTATION}(P, i)$ 
10:   $P' \leftarrow \text{APPLYPATCH}(P_i, S)$ 
11:  if  $\text{ISSYNTAXVALID}(P')$  and  $P' \neq S$  then
12:    return  $P'$ 
13:  end if
14: end for
15: return null {No valid adjustment found}

```

3.3 Patch Ranking

The patch ranking phase integrates reproduction test generation and patch prioritization to provide a comprehensive evaluation framework for selecting optimal repair solutions.

In the reproduction test generation process, KGCOMPASS utilizes a prompt that combines the issue description with 20 KG-mined relevant functions. Reproduction tests are specialized test cases dynamically generated by LLMs to simulate the exact conditions and environment, verifying whether a patch addresses the proposed issue. Using DeepSeek-V3 [8], KGCOMPASS iteratively generates tests that reproduce the described issue, averaging 113 valid reproduction tests per iteration and successfully generating at least one reproduction test for 203 (67.7%) instances in the SWE-bench Lite benchmark. All generated tests are executed within isolated Docker [10] containers to ensure testing consistency and reproducibility.

The patch prioritization process employs a multi-layer ranking strategy. For each syntactically valid patch, we evaluate four metrics in descending priority: (1) regression test passing count, strictly limited to only those tests that the original code already passed, (2) LLM-generated reproduction

test passing count, (3) majority voting (most frequently minimized LLM-generated patch), and (4) normalized patch size. The highest-ranked patch is selected as the final patch submission.

4 Experimental Setup

4.1 Benchmark

We evaluate KGCOMPASS on the repository-level repair benchmark SWE-bench Lite [16]. This benchmark contains 300 real GitHub issues from 12 active Python projects, together with full codebase snapshots and test suites. Each issue is presented in natural language, so the repair system must first identify the fault location within the repository and then generate a corresponding patch. SWE-bench Lite is the most popular repository-level repair benchmark currently, and it is more challenging than earlier benchmarks. Function-level repair benchmarks, such as TutorCode [43] and HumanEval-Java [15], isolate the buggy code in a single file. More complex benchmarks like Defects4J [17] and BugsInPy [40] still have some limitations. Bugs in Defects4J usually appear in a single file and have a median patch size of 4 lines. At the same time, BugsInPy provides an explicit failing test for every bug and has not been updated since 2020, which increases the risk of data leakage and causes frequent environment mismatches. In SWE-bench Lite, by contrast, a tool must bridge the semantic gap between an issue text and a codebase that can span thousands of files. For example, one snapshot of the Django project within this benchmark comprises 2,750 Python files and 27,867 functions, underscoring the scale of the fault location challenge. This mix of size, realism, and lack of location hints makes SWE-bench Lite the best available benchmark for modern, multi-file, issue-driven software repair.

4.2 Metrics

We follow standard evaluation practices for automated program repair and fault location [27, 33, 41, 46] and adopt a *top-1 success* protocol consistent with SWE-bench Lite evaluations [16]. We report four metrics: % **Resolved**, **File Acc.**, **Func Acc.**, and **Avg Cost** per bug. % Resolved is the fraction of instances whose top-ranked patch passes all ground-truth tests. File Acc. measures file-level fault location precision. Func Acc. measures function-level precision while resolving naming ambiguity. Avg Cost per bug captures computational efficiency under a fixed transparent workflow. Accurate fault location metrics remain a practical challenge [41, 46].

We compute per-instance overlaps with the Jaccard index. For files,

$$a_i^{\text{file}} = \frac{|F_i^* \cap \widehat{F}_i|}{|F_i^* \cup \widehat{F}_i|}. \quad (2)$$

For code elements, we unify functions and classes into one set,

$$a_i^{\text{func}} = \frac{|E_i^* \cap \widehat{E}_i|}{|E_i^* \cup \widehat{E}_i|}. \quad (3)$$

Averaged accuracies over the evaluated set S are

$$\text{FileAcc} = \frac{1}{|S|} \sum_{i \in S} a_i^{\text{file}}, \quad \text{FuncAcc} = \frac{1}{|S|} \sum_{i \in S} a_i^{\text{func}}. \quad (4)$$

S includes all instances within the repository-level repair benchmark SWE-bench Lite.

Jaccard-based formulation penalizes missing and spurious targets in a single score. Prior SWE-bench Lite fault location works typically report binary match rates at the file or function level rather than IoU [46]. IoU has been used to evaluate multi-location fault location in other settings, for example, concurrency fault location [11]. While successful fixes can occur at locations different

from the ground truth patch, overlap with ground truth patches yields stable metrics that correlate with repair accuracy [41, 46].

Importantly, “Func Acc.” can be lower than “Resolved”. The test-based success criterion tolerates extraneous edits that do not affect correctness. On SWE-bench Lite, the ground-truth patch edits one file, but a model may still modify additional functions within that file or repair the bug by changing locations that differ from the ground truth patch. Such displaced edits reduce the Jaccard overlap and therefore depress “File Acc.” and “Func Acc.”. In these cases, “File Acc.” and “Func Acc.” quantify fault location alignment with the ground truth patch, while “Resolved” captures end-to-end repair correctness.

4.3 Implementation Details

We use neo4j [29] for building the knowledge graph with plugins apoc-4.4 [31] and gds-2.6.8 [30]. KGCOMPASS integrates state-of-the-art LLMs and embedding models in a task-specific manner. We vary the repair model per experiment, defaulting to Claude-4 Sonnet [5] for fault location and patch generation, and to DeepSeek-V3 [8] for cost-efficient test generation. Our knowledge graph construction uses jina-embeddings-v2-base-code [14] for semantic embeddings, which was selected because it is a widely used open-source embedding model specifically designed for mixed natural language and code content in software repositories, which can accurately capture semantic relationships between issue descriptions and code entities. We empirically determined the hyperparameters through initial parameter exploration, setting $\beta = 0.6$ for path length decay and $\alpha = 0.3$ for the embedding-textual similarity balance in KGCOMPASS. We employ multiple temperature settings: 1.0 for test generation to promote diversity, 0 for deterministic LLM-based fault location, and temperatures 0 and 1.0 for mixture patch generation, following previous empirical studies [36, 52].

4.4 Baselines

We select the top ranked state-of-the-art repair systems on SWE-bench Lite leaderboard as baselines, including ExpeRepair [28], Refact.ai Agent [3], SWE-agent [45], DARS Agent [4], Lingxi [2], and OpenHands [39].

To isolate the effect of the knowledge graph in KGCOMPASS, we add an *LLM-only Fault Location* variant, named **pure-LLM**. It replaces the hybrid candidate selector in KGCOMPASS, which uses the knowledge graph’s top 15 functions plus up to 5 LLM suggestions, while the pure-LLM baseline proposes up to 20 function-level candidates from the issue description using the same fault location prompt (Figure 4). After candidate selection, all subsequent steps are identical to KGCOMPASS, isolating the effect of KG-based fault location and ensuring a fair comparison. This keeps the pipeline identical after the candidate step and yields a fair test of how KG-based fault location affects end-to-end repair.

4.5 Research Questions

- **RQ-1: How effective is KGCOMPASS in repository-level software repair compared to state-of-the-art approaches?** We measure repair success, fault location accuracy, and computational cost against (i) the pure-LLM baselines of Claude-4 Sonnet, Claude-3.5 Sonnet, DeepSeek-V3, and Qwen2.5 Max, and (ii) state-of-the-art open-source repair systems. Case studies further dissect KGCOMPASS’s strengths and remaining limitations.
- **RQ-2: How does the knowledge graph construction within KGCOMPASS contribute to fault location?** We analyze the effectiveness of our repository-aware knowledge graph in identifying relevant functions, examining multi-hop relationships, and path structures connecting issue descriptions to fault locations.

• **RQ-3: What are the impact of different components in KGCOMPASS on repair performance?**

Through ablation studies, we evaluate the contributions of key components, including candidate function selection strategies, entity path information in patch generation, various patch ranking approaches, and different candidate patch pool size configurations to understand their individual effects on repair success.

5 Experiments and Results

5.1 RQ-1: Effectiveness of KGCOMPASS

[Objective]: We aim to evaluate KGCOMPASS’s effectiveness in repository-level software repair by comparing its repair success, location accuracy, and cost against existing state-of-the-art approaches.

[Experimental Design]: The comparison baselines are detailed in Subsection 4.4. All experiments utilize the most popular repository-level repair benchmark, SWE-bench Lite [16], which has become the standard for repository-level repair due to its widespread adoption by recent state-of-the-art systems, ensuring fair and direct comparisons. For KGCOMPASS, we configure 4 backbone LLMs for patch generation, including Claude-4 Sonnet, Claude-3.5 Sonnet, DeepSeek-V3, and Qwen2.5 Max, according to Section 4.3. All systems are evaluated with four metrics: repair success (“Resolved”), file-level location accuracy (% *File Acc.*), function-level location accuracy (% *Func Acc.*), and average cost per bug (“Cost”), as detailed in Section 4.2. Furthermore, we perform cross-analysis on all these Claude-3.5 Sonnet and Claude-4 Sonnet based systems separately to determine whether KGCOMPASS can uniquely resolve errors that other systems cannot fix. Finally, we conduct an in-depth case study of a KGCOMPASS failed repair leveraging Claude-3.5 Sonnet.

[Experimental Results]: Across all tools, KGCOMPASS ranks third overall by % Resolved, is the top system with one single repair model, and is also the top single-LLM repair system that uses only Claude-3.5 Sonnet. Moreover, KGCOMPASS with Claude-4 Sonnet achieves the highest *Func Acc.* among all functions in Table 1. Based on Claude-3.5 Sonnet, KGCOMPASS resolves 46.0% of the 300 bugs, outperforming OpenHands (41.7%) and the pure-LLM baseline (35.3%) while keeping the average cost at \$0.2 per bug. The location precision of KGCOMPASS reaches 76.7% at the file level and 49.4% at the function level. With Claude-4 Sonnet as the repair model, KGCOMPASS attains 58.3% resolved with 83.6% file-level accuracy and 56.0% function-level accuracy, exceeding other single-LLM repair systems and yielding the highest function-level precision overall.

System	Repair Model	Resolved	Cost	File Acc.	Func Acc.
ExpeRepair	Claude-4 Sonnet & o4-mini	181 (60.3%)	~\$2.1	84.7	52.6
Refact.ai Agent	Claude-3.7 Sonnet & o4-mini	180 (60.0%)	N/A	80.6	47.0
KGCOMPASS	Claude-4 Sonnet	175 (58.3%)	\$0.2	83.6	56.0
SWE-agent	Claude-4 Sonnet	170 (56.7%)	~\$1.6	80.9	53.9
ExpeRepair	Claude-3.5 Sonnet & o3-mini	145 (48.3%)	\$2.1	80.7	49.6
SWE-agent	Claude-3.7 Sonnet	144 (48.0%)	~\$1.6	79.3	52.2
DARS Agent	Claude-3.5 Sonnet & DeepSeek-R1	141 (47.0%)	\$12.24	78.4	49.3
KGCOMPASS	Claude-3.5 Sonnet	137 (46.0%)	\$0.2	76.7	49.4
Lingxi	Claude-3.5 Sonnet	128 (42.7%)	N/A	66.1	39.7
OpenHands	Claude-3.5 Sonnet	125 (41.7%)	\$1.3	69.3	43.8
pure-LLM	Claude-4 Sonnet	116 (38.7%)	\$0.2	69.0	44.0
pure-LLM	Claude-3.5 Sonnet	106 (35.3%)	\$0.2	72.0	45.3

Table 1. SWE-bench Lite Results of Top-tier Repair Systems Without Fine-tuning

Figures 6 and 7 reveal both overlap and complementarity among the compared systems. Under the Claude-3.5 Sonnet setting, all 5 tools share 66 solved bugs. Beyond this overlap, KGCOMPASS contributes the largest number of unique fixes with 16, followed by ExpeRepair with 12, and DARS Agent, Lingxi, and OpenHands with 4 each. Under the Claude-4 Sonnet setting, the shared core is 119 solved bugs. Again, KGCOMPASS leads in exclusive coverage with 11 unique fixes, ahead of Refact.ai Agent with 7, SWE-agent with 5, and ExpeRepair with 4. These patterns show that the knowledge graph enables KGCOMPASS to expand coverage and handle cases that other state-of-the-art systems miss.

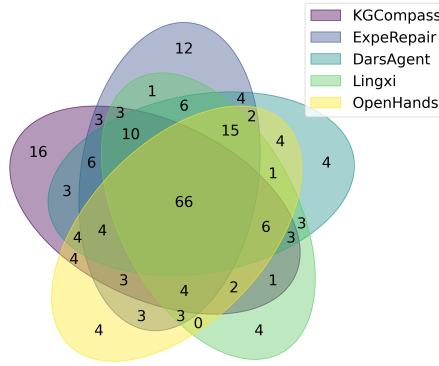


Fig. 6. Intersection Analysis of KGCOMPASS Against Leading Claude-3.5 based Systems

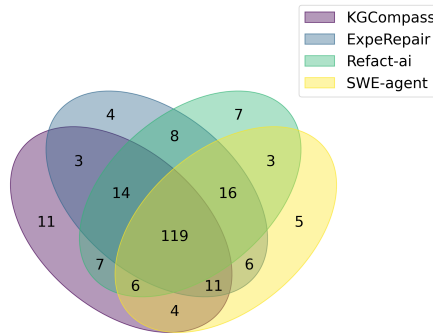


Fig. 7. Intersection Analysis of KGCOMPASS Against Leading Claude-4 based Systems

Table 2 shows that KGCOMPASS improves on every repair model. Compared with corresponding pure-LLM baselines, the resolved rate rises by 50.8% on Claude-4 Sonnet, 30.2% on Claude-3.5 Sonnet, 115.7% on DeepSeek-V3, and 156.4% on Qwen2.5 Max. File- and function-level location accuracy increases by up to 73.7% and 139.2%, respectively. These consistent gains indicate that the KG-enhanced repair workflow of KGCOMPASS demonstrates generalizability across varying backbone LLMs.

Table 3 reports an average cost of \$0.1999 per bug. The patch phase dominates the budget at \$0.1730, split between the input prompt (\$0.1226) and the model’s response (\$0.0504). The fault location process costs \$0.0080, primarily due to the use of candidates generated by the KG, enabling

System	Base Model	Resolved	File Acc.	Func Acc.
KGCOMPASS	Claude-4 Sonnet	58.3%	83.6	56.0
pure-LLM	Claude-4 Sonnet	38.7%	69.0	44.0
KGCOMPASS	Claude-3.5 Sonnet	46.0%	76.7	49.4
pure-LLM	Claude-3.5 Sonnet	35.3%	72.0	45.3
KGCOMPASS	DeepSeek-V3	36.7%	76.3	43.7
pure-LLM	DeepSeek-V3	17.0%	54.0	27.0
KGCOMPASS	Qwen2.5 Max	33.3%	74.7	48.6
pure-LLM	Qwen2.5 Max	13.0%	43.0	20.3

Table 2. SWE-bench Lite Results of KGCOMPASS vs. Pure-LLM of 4 Repair LLMs.

the LLM to provide additional supplementary candidate locations. The repair process for each instance only needs to generate them once. Reproduction tests add \$0.0189 thanks to the lower price of DeepSeek-V3.

Stage	Tokens	Price \$ / M Tokens	Cost
Patch - Input	40880.0	\$3.00	\$0.1226
Patch - Output	3360.9	\$15.00	\$0.0504
Location - Input	470.8	\$3.00	\$0.0014
Location - Output	441.4	\$15.00	\$0.0066
Tests - Input	63455.7	\$0.28	\$0.0175
Tests - Output	2555.9	\$0.55	\$0.0014
Total per Bug			\$0.1999

Table 3. Token and Cost Statistics of KGCOMPASS

As a stratified analysis across Claude-3.5 Sonnet and Claude-4 Sonnet, splitting instances by whether the issue text contains an explicit file or function name shows larger gains when no hints are present and a stronger lift with Claude-4 (Table 4). With Claude-3.5, KGCompass exceeds the pure-LLM baseline by 18.5% with hints (65.3% vs 55.1%) and by 42.4% without hints (36.6% vs 25.7%). With Claude-4, the advantage widens to 33.7% with hints (76.5% vs 57.2%) and 69.5% without hints (49.5% vs 29.2%). Weighted by subset shares, these splits reproduce the headline resolved rates and align with the multi-hop patterns in Subsection 5.2, where most successful localizations depend on indirect links captured by the knowledge graph.

[Failed Case Study]: Instance `django-12589` shows how the knowledge graph helps when an issue’s text provides no location hint. The bug is an ambiguous “GROUP BY” clause that triggers a runtime SQL error in Django 3.0. Pure LLM systems cannot even localize the fault, but KGCOMPASS follows the path `issue` → `query.py` → `set_group_by` and ranks the target function first. This two-hop trace combines an issue entity, a file entity, and a function entity, a type of link that text-only analysis cannot obtain. With Claude-3.5 Sonnet, KGCOMPASS produced a near-miss patch because the LLM lacked framework-specific knowledge in Django’s ORM. Using the same KG context but switching the repair model to Claude-4 Sonnet, KGCOMPASS successfully repaired this instance. This supports the view that once the knowledge graph disambiguates location, stronger pretraining improves patch generation.

Table 4. Direct-hint vs no-hint performance on SWE-bench Lite by backbone. Baseline is the pure-LLM with the same backbone.

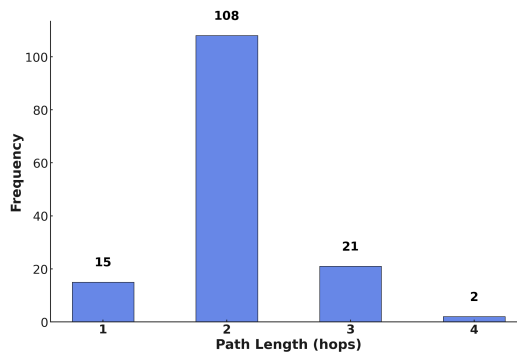
Subset	Claude-3.5 Sonnet		Claude-4 Sonnet	
	KGCompass	pure-LLM	KGCompass	pure-LLM
With explicit location (32.7%)	65.3%	55.1%	76.5%	57.2%
Without explicit location (67.3%)	36.6%	25.7%	49.5%	29.2%

[RQ-1] Findings: (1) KGCOMPASS ranks third overall by % Resolved, is the best among state-of-the-art systems with a single repair model, and is also the best among systems that use only Claude-3.5 as the repair model. KGCOMPASS further achieves the highest function-level location accuracy across all systems while keeping cost at \$0.2 per bug. (2) KGCOMPASS yields large relative gains over pure-LLM baselines across backbones, +50.8% on Claude-4 Sonnet, +30.2% on Claude-3.5 Sonnet, +115.7% on DeepSeek-V3, and +156.4% on Qwen2.5 Max. (3) KGCOMPASS uniquely fixes the most (11) bugs under Claude-4 and most (16) bugs under Claude-3.5 Sonnet, providing complementary coverage beyond existing repair systems. **Insights:** (1) Leveraging a repository-aware knowledge graph to link repository artifacts and codebase entities narrows the search space and boosts repair accuracy at low cost. (2) Upgrading the repair model’s pretraining capacity further improves patch generation once the knowledge graph resolves the fault location.

5.2 RQ-2: Contribution of Knowledge Graph to fault location

[Objective]: We aim to assess the impact of the knowledge graph on fault location by measuring its ability to (i) cover ground-truth locations and (ii) rank candidate functions accurately, thereby clarifying the knowledge graph’s role in KGCOMPASS’s overall effectiveness.

[Experimental Design]: To assess the knowledge graph in isolation, we analyze the pure KG-mined 20 candidate bug functions, which means the list returned by the knowledge graph without any LLM-generated fault locations. First, we measure the number of ground-truth locations that fall within this set to establish coverage. Second, we examine the hop count from the starting issue node to its ground-truth function to see how often multi-hop links are needed. Third, we break down the intermediate node types on these paths to illustrate how code entities and repository artifacts are connected. Finally, we record the rank of every ground-truth function within the KG-mined 20 list to evaluate the accuracy of our relevance function (Equation 1).

**Fig. 8.** Path Length Distribution for Ground Truth fault locations

[Experimental Results]: As shown in Table 5, KG-mined 20 relevant functions cover 68.7% of file-level and 40.4% of function-level ground truth fault locations, without leveraging LLMs. Figure 8 illustrates the hop distribution of the ground truth patch from the starting issue within the knowledge graph, reveals that only 15 of the 146 ground-truth bug functions (10.3%) are directly connected to origin issue descriptions, whereas 108 (74.0%), 21 (14.4%), and 2 (1.4%) functions lie two, three, and four hops away, yielding an aggregate 89.7% that depend on at least one intermediate entity. Some ground-truth patches modified more than one function. These statistics demonstrate that effective repository-level fault location benefits mostly from exploiting the multi-hop relations captured in the knowledge graph, since pure text-based LLM workflows display substantial model-dependent variability in accuracy, thereby validating KGCOMPASS’s knowledge graph-based design.

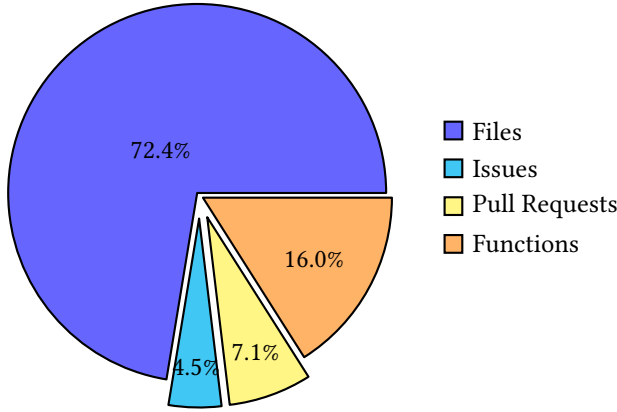


Fig. 9. Intermediate Entity Types in KG Paths to Ground Truth Locations

Figure 9 further details the 156 intermediate entities observed on ground truth paths: 113 files (72.4%), 25 functions (16.0%), 11 pull requests (7.1%), and 7 issues (4.5%). The dominance of files and functions underscores the importance of static structural links mined by KG within the codebase. Meanwhile, the presence of issues and pull requests (11.6%) demonstrates that repository artifacts provide essential context for bridging origin issue reports to the codebase. This distribution justifies the design of KGCOMPASS to combine static analysis with repository artifacts in the knowledge graph for reliable location.

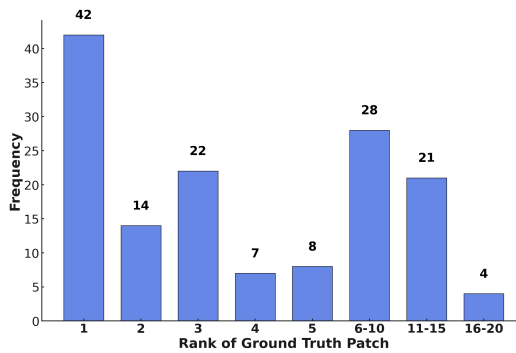


Fig. 10. Rank of Ground Truth Patch in KG Candidates

Figure 10 plots the rank of each ground-truth function among the 20 KG-mined candidates. 42 functions (28.8%) are placed at rank 1, 93 (63.7%) fall within the first five positions, and 121 (82.9%)

appear in the top 10. There are only 4 instances beyond rank 15. The strong concentration near the top confirms the relevance score’s effectiveness. In contrast, the small tail indicates that additional ranking signals are still required, thereby justifying our hybrid KG + LLM location strategy. 15 KG-mined functions with the highest relevant score hit cover 97.3% of the obtained ground-truth locations. Therefore, we retain 15 from KG plus 5 more from LLM to provide mixed candidate function locations.

[RQ-2] Findings: (1) Only 10.3% of bugs sit in a 1-hop relationship from the origin issue, whereas 89.7% require 2 or more hops; accurate location, therefore, relies on traversing indirect links encoded in the knowledge graph. (2) Although files and functions form 88.4% of intermediate nodes, the remaining 11.6% are issues and pull requests, indicating that repository artifacts supply critical context unavailable from static code alone. (3) Within 20 KG-mined candidates, 82.9% of ground-truth functions appear in the first 10, showing that the relevance score produces a compact, high-quality set that the hybrid KG + LLM pipeline can further refine. **Insights:** By integrating the structural codebase with repository artifacts, the knowledge graph bridges the semantic gap between issue reports and buggy code, enabling reliable fault location.

5.3 RQ-3: Impact of Components on KGCOMPASS’s Repair Performance

[Objective]: We measure the contribution of three key components in KGCOMPASS, namely KG-based fault location, entity-path prompting, and the pool size of candidate patches.

[Experimental Design]: We run ablations that vary one factor at a time. All variants share the same configuration for the main pipeline: Claude-3.5 Sonnet for location and patch generation, DeepSeek-V3 for reproduction tests, identical prompt templates, and fixed temperature settings. Any change in outcome is thus attributable to the component under examination. We compare three location schemes (KG-only, LLM-only, and the hybrid KG+LLM), each restricted to 20 functions. We then test patch generation with and without KG-derived entity-path context, while fixing the candidate list and using temperature 0.0. Next, we evaluate a cumulative ranking pipeline that starts with majority voting and successively adds regression tests, reproduction tests, and a patch-size tie-breaker, keeping previously introduced criteria. Finally, with the best ranking policy fixed, we vary the patch pool size (1, 2, 4, 6) to study the diversity–quality trade-off. In addition to the Claude-3.5 pipeline, we report Claude-4 results under the same settings to validate effectiveness and generalizability.

[Experimental Results]: *Fault Location.* With Claude-4, the hybrid KG+LLM locator achieves 91.0% file-level and 66.7% function-level coverage, compared to LLM-only at 77.3% and 52.6%, respectively (Table 5). Claude-3.5 shows the same pattern: hybrid reaches 84.3% file-level and 57.7% function-level coverage, exceeding LLM-only at 81.0% and 53.1%, and KG-only at 68.7% and 40.4%. These results confirm that KG structural links and LLM semantic reasoning are complementary.

Strategy	% GT File	% GT Function
KG	68.7	40.4
Claude-3.5 Sonnet	81.0	53.1
KG + Claude-3.5 Sonnet	84.3	57.7
Claude-4 Sonnet	77.3	52.6
KG + Claude-4 Sonnet	91.0	66.7

Table 5. Performance Between LLM-based and Hybrid Candidate Selection Strategies

Entity-path prompting. At temperature 0.0 with a fixed candidate list, Claude-4 improves from 91 resolved bugs (30.3%) without the KG to 131 (43.7%) with KG but without entity paths, and to 142 (47.3%) when entity paths are included (Table 6). Claude-3.5 shows the same trend, rising from 85 (28.3%) without KG to 102 (34.0%) without paths and to 108 (36.0%) with paths. The motivating example illustrates why the path signal matters: only when the prompt contained the *issue* $\rightarrow$ *PR* $\rightarrow$ *_print_MatAdd* path did the model produce the robust `getattr`-based patch.

Model	Variant	Resolved
Claude-3.5 Sonnet	KGCOMPASS	108 (36.0%)
	w/o KG	85 (28.3%)
	w/o Entity Path	102 (34.0%)
Claude-4 Sonnet	KGCOMPASS	142 (47.3%)
	w/o KG	91 (30.3%)
	w/o Entity Path	131 (43.7%)

Table 6. Ablation Study of KG and Relationship Path Information within KGCOMPASS, with Temperature 0.0

Ranking pipeline. Strengthening the selector yields large gains and approaches the oracle ceiling. For Claude-4, accuracy rises from 139 (46.3%) with greedy sampling to 150 (48.3%) with majority voting, 164 (51.7%) after adding regression tests, and 175 (58.3%) after adding reproduction tests, while the upper bound is 182 (60.7%) (Table 7). Claude-3.5 follows the same progression, from 108 (36.0%) to 121 (40.3%), then 129 (43.0%), and finally 138 (46.0%), close to its 143 (47.7%) ceiling. On Claude-4, the ground-truth upper bound reaches 182 solved bugs (60.7%), surpassing ExpeRepair’s 181 (60.3%) and **ranking first** among the compared systems.

Ranking Strategy	Claude-3.5	Claude-4
Greedy Sampling	108 (36.0%)	139 (46.3%)
Majority Voting	121 (40.3%)	150 (48.3%)
+ Regression tests	129 (43.0%)	164 (51.7%)
+ Reproduction tests	138 (46.0%)	175 (58.3%)
Ground-truth tests	143 (47.7%)	182 (60.7%)

Table 7. Ablation Study of Patch Ranking Strategies, with 6 Candidate Patches

Size of Patch Pool. Increasing the patch pool size can improve repair accuracy. With Claude-4, patch pool size with 1, 2, 4, and 6 achieve 142 (47.3%), 148 (49.3%), 161 (53.7%), and 175 (58.3%), respectively. Claude-3.5 shows the same saturation, moving from 108 (36.0%) to 116 (38.7%), 126 (42.0%), and 138 (46.0%). A larger patch pool size would likely close that gap, but it would also increase prompt length and evaluation time, driving costs up. We therefore cap the pool at 6 candidates for KGCOMPASS. As inference cost drops in the future, raising the patch pool size could be a practical way to unlock the small remaining performance margin.

Candidate Patches	Claude3.5		Claude4	
	KGCOMPASS	Upper Bound	KGCOMPASS	Upper Bound
1	108 (36.0%)	108 (36.0%)	142 (47.3%)	142 (47.3%)
2	116 (38.7%)	125 (41.7%)	148 (49.3%)	151 (50.3%)
4	126 (42.0%)	136 (45.3%)	161 (53.7%)	167 (55.7%)
6	138 (46.0%)	143 (47.7%)	175 (58.3%)	182 (60.7%)

Table 8. Effect of Candidate Patch Pool Size on KGCompass Repair Accuracy

[RQ-3] Findings: (1) The hybrid KG+LLM locator delivers the highest coverage. With Claude-4, it reaches 91.0% file-level and 66.7% function-level, and with Claude-3.5, it reaches 84.3% and 57.7%. (2) KG-derived entity paths improve repair success beyond KG-driven location alone, from 131 to 142 resolved on Claude-4 and from 102 to 108 on Claude-3.5 at temperature 0.0. (3) The layered ranking pipeline raises success to 175 (58.3%) on Claude-4 and 138 (46.0%) on Claude-3.5, both close to their upper bounds. (4) Leveraging the ground-truth upper bound, KGCOMPASS with Claude-4 reaches 182 (60.7%), higher than ExpeRepair’s 181 (60.3%), indicating meaningful headroom for stronger ranking. **Insights:** (1) Knowledge graph not only improves location, but also guides the LLM’s patch generation through entity paths; the same idea could be applied to other LLM-based tasks. (2) Patch selection strongly affects final repair success; systematic yet lightweight ranking rules already approach the upper limit, so future work on learning-based or more robust rankers may close the remaining performance gap without manual design and heavy cost.

6 Threats to Validity

Internal Validity. Internal validity concerns potential experimental biases that could affect the fairness of the results. To prevent this, we enforced strict temporal constraints during our knowledge graph construction, incorporating only issues and pull requests that existed before each issue’s creation time within SWE-bench Lite, thereby preventing data contamination from future artifacts. Similarly, for patch evaluation, we used only regression tests that passed successfully in the original codebase before the bug was reported, ensuring a fair assessment of repair capabilities.

External Validity. External validity addresses how far our conclusions extend beyond the Python projects in SWE-Bench Lite. KGCOMPASS mitigates this threat by keeping language-specific logic in thin adapter layers. Each language implements a lightweight `LanguageConfig` and a corresponding `Parser`, while graph construction, ranking, and repair modules remain unchanged. Adding support for other programming languages, therefore, requires only modest, self-contained code, leaving the core workflow intact. This modular design reduces dependence on any single language and facilitates adoption in other systems.

Construct Validity. Construct validity concerns whether we accurately measure what we claim to evaluate. To address this, we employed multiple complementary metrics: repair success rate (“Resolved”), fault location accuracy at file and function levels (“File Acc.” and “Func Acc.”), and computational efficiency (“Avg Cost”). By comparing KGCOMPASS’s generated patches with ground truth patches, we provided validation beyond test-passing success, helping identify truly correct fixes rather than merely coincidental solutions that happen to pass tests.

7 Related Work

7.1 Repository-level Software Repair

Recent large language models (LLMs) have demonstrated remarkable capabilities in repository-level software repair tasks [39, 41, 44, 47]. SWE-Bench Lite [16] provides a representative benchmark of 300 real-world GitHub issues, making it suitable for evaluating LLM-based software repair approaches. Repository-level repair approaches fall into two categories: **Agentic approaches** coordinate multiple specialized agents through observe-think-act loops [1, 18, 38, 39, 45, 47]. OpenHands [39] employs agents interacting with sandbox environments, while AutoCodeRover [38, 47] leverages hierarchical code search for repository navigation. Composio [1] implements a state-machine multi-agent system using LangGraph [19], and SWE-agent [45] introduces specialized tools for LLM interactions. **Procedural approaches** employ expert-designed sequential workflows for repair [6, 12, 41, 47]. Agentless [41] employs a three-phase repair, including fault location, patch generation, and patch validation. Moatless [6, 32] proposed that, rather than relying on an agent to reason its way to a solution, it is crucial to build good tools to insert the right context into the prompt and handle the response.

Pure LLM-based approaches treat repository artifacts and code as separate entities, rather than as interconnected components, which hinders their ability to bridge the gap between natural language issues and the structured codebase. In contrast, KGCOMPASS explicitly models both the structural codebase and the semantic relationships across repository artifacts through knowledge graphs.

7.2 Knowledge Graph for Repository-level Software

Knowledge graphs have emerged as a powerful approach for modeling code repositories by capturing complex relationships between code entities and their dependencies [22, 23, 33, 49]. Recent work has demonstrated the effectiveness of knowledge graphs in understanding software repositories. RepoGraph [33] operates at a fine-grained code line level, demonstrating that modeling explicit definition-reference relationships can effectively guide repair decisions. GraphCoder [22] introduces control flow and dependency analysis into code graphs, showing that structural code relationships are critical for understanding code context and predicting the subsequent statements in completion tasks. CodeXGraph [23] represents code symbols and their relationships in graph databases, proving that querying graph structures enables more precise retrieval of relevant code snippets than traditional sequence-based approaches.

Existing graph-based approaches primarily focus on code structure, overlooking key **repository artifacts** (such as issues and pull requests), and fail to **leverage indirect entity relationships** for patch generation. KGCOMPASS integrates repository artifacts, combining codebase entities, into the knowledge graph and introduces path-guided repair, strengthening the link between issues and relevant functions and enabling more precise repairs.

8 Conclusion

This paper introduces KGCOMPASS, a repository-aware repair framework that constructs a knowledge graph linking issues, pull requests, files, and functions, then steers a large language model along ranked multi-hop paths to generate patches. On the SWE-Bench Lite benchmark, KGCOMPASS repairs 58.3% of bugs and achieves 83.6% file-level and 56.0% function-level fault location at an average cost of \$0.2 per instance, outperforming state-of-the-art single-LLM baselines, and boosting repair success by 30.2% to 156.4% across four backbone LLMs. The graph reduces the search space from thousands to 20 candidates, addressing the context-length constraint that limited repository-level repair. 89.7% of successful fault locations require 2+ hops; therefore, the knowledge graph successfully bridges the semantic gap between natural-language bug reports and codebase by

capturing indirect structural relations that text-only LLMs miss. Path-guided prompting attaches an explicit reasoning chain to each candidate, and ablations show that omitting it sharply lowers accuracy. Supplying the language model with this compact, context-rich candidate set further stabilizes repair quality and reduces inference cost, thereby mitigating the model-dependent variability observed in pure LLM workflows. Future work could enrich the knowledge graph with domain-specific knowledge, and could extend the path-guided paradigm to multi-hop question answering, fact verification, and other natural-language reasoning tasks.

References

- [1] 2024. Composio.dev. <https://composio.dev/>
- [2] 2025. <https://github.com/nimasteryang/Lingxi> Accessed: 2025-09-10.
- [3] 2025. refactor.ai - ai coding agent for software development. <https://refactor.ai/> Accessed: 2025-09-10.
- [4] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. DARS: Dynamic Action Re-Sampling to Enhance Coding Agent Performance by Adaptive Tree Traversal. *arXiv:2503.14269* [cs.CL] <https://arxiv.org/abs/2503.14269>
- [5] Anthropic. 2025. Introducing Claude 4. <https://www.anthropic.com/news/claude-4> Accessed: 2025-09-10.
- [6] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2024. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. *arXiv preprint arXiv:2410.20285* (2024).
- [7] Andrea Ballatore, Michela Bertolotto, and David C Wilson. 2015. A structural-lexical measure of semantic similarity for geo-knowledge graphs. *ISPRS International Journal of Geo-Information* 4, 2 (2015), 471–492.
- [8] DeepSeek-AI et al. 2024. DeepSeek-V3 Technical Report. *arXiv:2412.19437* [cs.CL] <https://arxiv.org/abs/2412.19437>
- [9] Edsger W Dijkstra. 1959. A note on two problems in connexion with graphs.
- [10] Docker. [n. d.]. Docker - Build, Share, and Run Your Applications. <https://www.docker.com/> Accessed: 2025-03-01.
- [11] Zuo Cheng Feng, Kaiwen Zhang, Miaomiao Wang, Yiming Cheng, Yuandao Cai, Xiaofeng Li, and Guan Jun Liu. 2025. Deep Learning Based Concurrency Bug Detection and Localization. *arXiv preprint arXiv:2508.20911* (2025).
- [12] Anmol Gautam, Kishore Kumar, Adarsh Jha, Mukunda NS, and Ishaan Bhola. 2024. SuperCoder2. 0: Technical Report on Exploring the feasibility of LLMs as Autonomous Programmer. *arXiv preprint arXiv:2409.11190* (2024).
- [13] Paul Gauthier. 2024. Aider is ai pair programming in your terminal.
- [14] Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2024. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. *arXiv:2310.19923* [cs.CL] <https://arxiv.org/abs/2310.19923>
- [15] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1430–1442. [doi:10.1109/ICSE48619.2023.00125](https://doi.org/10.1109/ICSE48619.2023.00125)
- [16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [17] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. [doi:10.1145/2610384.2628055](https://doi.org/10.1145/2610384.2628055)
- [18] Kodu. 2024. Kodu.ai. <https://www.kodu.ai/>
- [19] LangChain. 2024. LangGraph: Manage LLM agent state and logic as a graph. <https://www.langchain.com/langgraph> Accessed: 2025-03-01.
- [20] Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025. PatchPilot: A Stable and Cost-Efficient Agentic Patching Framework. *arXiv preprint arXiv:2502.02747* (2025).
- [21] Kui Liu, Shangwen Wang, Anil Koyuncu, Kisub Kim, Tegawendé F Bissyandé, Dongsun Kim, Peng Wu, Jacques Klein, Xiaoguang Mao, and Yves Le Traon. 2020. On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 615–627.
- [22] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-fine Retrieval Based on Code Context Graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 570–581.

- [23] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. 2024. Codexgraph: Bridging large language models and code repositories via code graph databases. *arXiv preprint arXiv:2408.03910* (2024).
- [24] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899* (2024).
- [25] Yu Liu, Sergey Mechtaev, Pavle Subotić, and Abhik Roychoudhury. 2023. Program repair guided by datalog-defined static analysis. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1216–1228.
- [26] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. 2024. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* (2024).
- [27] Zexiong Ma, Shengnan An, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2024. Repository Structure-Aware Training Makes SLMs Better Issue Resolver. *arXiv preprint arXiv:2412.19031* (2024).
- [28] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2025. EXPEREPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. *arXiv preprint arXiv:2506.10484* (2025).
- [29] Neo4j. [n. d.]. Neo4j | Graph Database Platform. <https://neo4j.com/> Accessed: 2025-03-01.
- [30] Neo4j. 2024. Neo4j Graph Data Science. <https://github.com/neo4j/graph-data-science>. Accessed: 2025-03-01.
- [31] Neo4j Contributors. 2025. Awesome Procedures for Neo4j 4.4.x. <https://github.com/neo4j-contrib/neo4j-apoc-procedures>. Accessed: 2025-03-01.
- [32] Albert Örwall. 2024. Moatless Tools. <https://github.com/aorwall/moatless-tools>.
- [33] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2024. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint arXiv:2410.14684* (2024).
- [34] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl Barr, and Sergey Mechtaev. 2024. The Fact Selection Problem in LLM-Based Program Repair. arXiv:2404.05520 [cs.SE] <https://arxiv.org/abs/2404.05520>
- [35] PatchKitty. 2024. PatchKitty-0.9. https://github.com/SWE-bench/experiments/tree/main/evaluation/lite/20241220_PatchKitty-0.9_claude-3.5-sonnet-20241022 Accessed: 2025-01-01.
- [36] Ruizhong Qiu, Weiliang Will Zeng, James Ezick, Christopher Lott, and Hanghang Tong. 2024. How efficient is LLM-generated code? A rigorous & high-standard benchmark. *arXiv preprint arXiv:2406.06647* (2024).
- [37] Benjamin Rombaut, Sogol Masoumzadeh, Kirill Vasilevski, Dayi Lin, and Ahmed E Hassan. 2024. Watson: A Cognitive Observability Framework for the Reasoning of Foundation Model-Powered Agents. *arXiv preprint arXiv:2411.03455* (2024).
- [38] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2024. Specrover: Code intent extraction via llms. *arXiv preprint arXiv:2408.02232* (2024).
- [39] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] <https://arxiv.org/abs/2407.16741>
- [40] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, et al. 2020. Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1556–1560.
- [41] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [42] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *arXiv preprint arXiv:2506.23749* (2025).
- [43] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 882–894. doi:10.1145/3650212.3680328
- [44] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawendé F. Bissyandé, Claire Le Goues, and Shunfu Jin. 2024. Multi-Objective Fine-Tuning for Enhanced Program Repair with LLMs. arXiv:2404.12636 [cs.SE] <https://arxiv.org/abs/2404.12636>
- [45] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://arxiv.org/abs/2405.15793>

- [46] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. 2025. OrcaLoca: An LLM Agent Framework for Software Issue Localization. *arXiv preprint arXiv:2502.00350* (2025).
- [47] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
- [48] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.
- [49] Yanjie Zhao, Haoyu Wang, Lei Ma, Yuxin Liu, Li Li, and John Grundy. 2019. Knowledge graphing git repositories: A preliminary study. In *2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 599–603.
- [50] Ganggao Zhu and Carlos A Iglesias. 2016. Computing semantic similarity of concepts in knowledge graphs. *IEEE Transactions on Knowledge and Data Engineering* 29, 1 (2016), 72–85.
- [51] Qihao Zhu, Qingyuan Liang, Zeyu Sun, Yingfei Xiong, Lu Zhang, and Shengyu Cheng. 2024. GrammarT5: Grammar-integrated pretrained encoder-decoder neural model for code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [52] Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Zhi Jin, and Hong Mei. 2024. Hot or cold? adaptive temperature sampling for code generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 437–445.